

Chapter 4.7: Data Hazards

Forwarding versus Stalling

Contents

1	Introduction	2
1.1	A Sequence with Many Dependences	2
2	Visualizing Pipeline Data Hazards (Figure 4.50)	2
3	Forwarding (Bypassing)	2
3.1	Hazard Conditions	3
3.2	Avoiding Incorrect Forwarding	3
4	Forwarding Hardware (Figure 4.52)	3
5	Forwarding Control Values (Figure 4.53)	4
5.1	Forwarding Logic Conditions	4
5.2	Double Data Hazard	4
6	Data Hazards and Stalls	5
6.1	Hazard Detection Unit	5
6.2	How Stalls Work	5
7	The Big Picture	6
8	Flashcards	7

1 Introduction

“What do you mean, why’s it got to be built? It’s a bypass. You’ve got to build bypasses.”
— Douglas Adams, *The Hitchhiker’s Guide to the Galaxy*, 1979

The examples in the previous section showed the power of pipelined execution with **independent** instructions. Now it’s time to take off the rose-colored glasses and look at what happens with **real programs** that have **data dependences**.

1.1 A Sequence with Many Dependences

Consider this RISC-V code sequence where the last four instructions all depend on the result of the first instruction (x2):

```
sub  x2, x1, x3    // Register x2 written by sub
and  x12, x2, x5   // 1st operand (x2) depends on sub
or   x13, x6, x2   // 2nd operand (x2) depends on sub
add  x14, x2, x2   // 1st & 2nd (x2) depend on sub
sd   x15, 100(x2)  // Base (x2) depends on sub
```

If register x2 had the value **10** before the **sub** instruction and **−20** afterwards, the programmer intends that **−20** will be used in all following instructions that refer to x2.

2 Visualizing Pipeline Data Hazards (Figure 4.50)

The diagram below shows how the five instructions execute over time. Register x2 is **written in clock cycle 5** (WB stage of **sub**), so the proper value is unavailable before CC5.

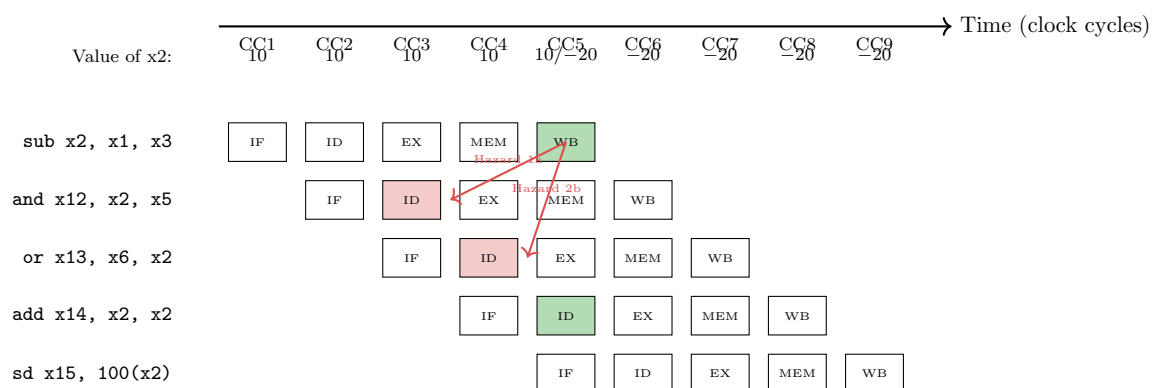


Figure 1: **Figure 4.50:** Pipelined dependences. The **and** and **or** instructions read x2 *before* it is written (red arrows go backward in time ⇒ **data hazards**).

Key Insight

- **and** reads x2 in CC3 (ID stage), but **sub** writes x2 in CC5 (WB) ⇒ **Hazard 1a**.
- **or** reads x2 in CC4 (ID stage), but **sub** writes x2 in CC5 (WB) ⇒ **Hazard 2b**.
- **add** reads x2 in CC5 (ID stage) ⇒ **No hazard** (register file internal forwarding).
- **sd** reads x2 in CC6 ⇒ **No hazard**.

3 Forwarding (Bypassing)

The desired result is available at the **end of the EX stage** of the **sub** instruction (CC3). The **and** and **or** instructions need the data at the **beginning of their EX stage** (CC4 and CC5 respectively).

Solution: We can “forward” the data as soon as it is available to any units that need it, *before* it is written to the register file.

3.1 Hazard Conditions

A notation names the fields of the pipeline registers:

- ID/EX.RegisterRs1: The first source register number in the ID/EX pipeline register.
- EX/MEM.RegisterRd: The destination register number in the EX/MEM pipeline register.

The two pairs of hazard conditions are:

Type	Condition
1a	EX/MEM.RegisterRd = ID/EX.RegisterRs1
1b	EX/MEM.RegisterRd = ID/EX.RegisterRs2
2a	MEM/WB.RegisterRd = ID/EX.RegisterRs1
2b	MEM/WB.RegisterRd = ID/EX.RegisterRs2

Example: Classifying the Dependences

In the sequence from above:

- sub-and is a **Type 1a** hazard: EX/MEM.RegisterRd = ID/EX.RegisterRs1 = x2.
- sub-or is a **Type 2b** hazard: MEM/WB.RegisterRd = ID/EX.RegisterRs2 = x2.
- sub-add: **Not a hazard** (register file supplies data during ID stage of add).
- sub-sd: **Not a hazard** (sd reads x2 the cycle *after* sub writes it).

3.2 Avoiding Incorrect Forwarding

Some instructions do not write registers. We must check:

1. The RegWrite signal is active.
2. The destination register is **not** x0 (since x0 must always be 0).

4 Forwarding Hardware (Figure 4.52)

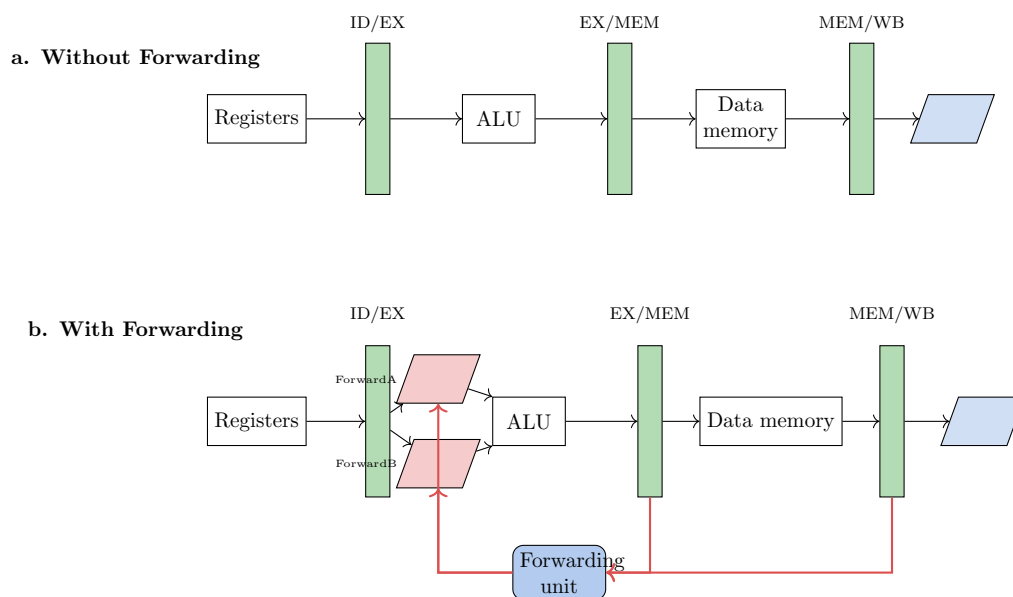


Figure 2: **Figure 4.52:** (a) ALU and pipeline registers *without* forwarding. (b) With forwarding: multiplexers added to ALU inputs, controlled by the Forwarding Unit.

5 Forwarding Control Values (Figure 4.53)

Mux Control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

Table 1: **Figure 4.53:** Control values for the forwarding multiplexors.

5.1 Forwarding Logic Conditions

EX Hazard (Forward from EX/MEM)

```

if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
    ForwardA = 10

if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
    ForwardB = 10

```

MEM Hazard (Forward from MEM/WB)

```

if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd != 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
    ForwardA = 01

if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd != 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
    ForwardB = 01

```

5.2 Double Data Hazard

Consider:

```

add x1, x1, x2
add x1, x1, x3
add x1, x1, x4

```

Both the first and second instructions write to **x1** before the third instruction reads it. The result should be forwarded from the **MEM stage** (most recent). The MEM hazard condition must check that there is **NOT** a more recent EX hazard.

Revised MEM Hazard Logic (Double Data Hazard)

The key insight: only forward from MEM/WB if there is **NOT** a more recent EX hazard.

```
// MEM hazard for Rs1 (with double hazard check)
if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd != 0)
    and NOT(EX/MEM.RegWrite
        and (EX/MEM.RegisterRd != 0)
        and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
    ForwardA = 01

// MEM hazard for Rs2 (with double hazard check)
if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd != 0)
    and NOT(EX/MEM.RegWrite
        and (EX/MEM.RegisterRd != 0)
        and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
    ForwardB = 01
```

The EX hazard conditions remain unchanged:

```
// EX hazard (always takes priority - most recent value)
if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
    ForwardA = 10

if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
    ForwardB = 10
```

6 Data Hazards and Stalls

Load-Use Hazard: Forwarding Cannot Help!

When a load instruction is immediately followed by an instruction that uses its result, forwarding **cannot** save the day. The data is still being read from memory in CC4 (MEM stage) while the dependent instruction needs it at the *beginning* of CC4 (EX stage).

6.1 Hazard Detection Unit

The hazard detection unit operates during the **ID stage** to insert a stall:

Load-Use Hazard Detection

```
if (ID/EX.MemRead
    and ((ID/EX.RegisterRd = IF/ID.RegisterRs1)
        or (ID/EX.RegisterRd = IF/ID.RegisterRs2)))
    stall the pipeline
```

6.2 How Stalls Work

When stalling:

1. **Prevent PC** from updating (re-fetch the same instruction).
2. **Prevent IF/ID register** from changing (re-decode the same instruction).

3. **Insert a bubble (NOP)** by setting all control signals in ID/EX to 0.

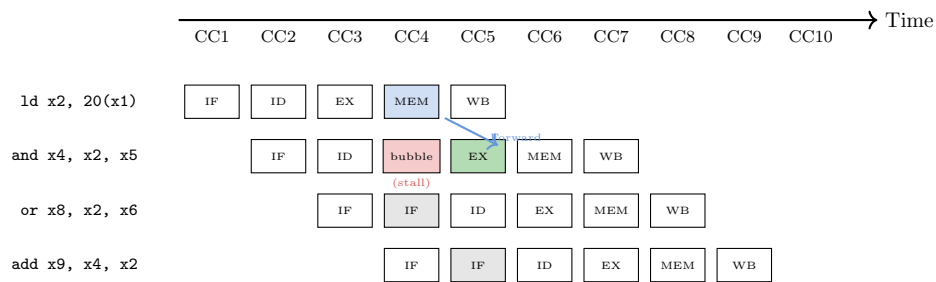


Figure 3: **Figure 4.57:** How stalls are inserted. The **and** instruction (dependent on **ld**) gets a **bubble** inserted at CC4. The PC and IF/ID registers are frozen, causing subsequent instructions to repeat their IF stage. After the 1-cycle stall, forwarding from MEM/WB to EX handles the dependence.

7 The Big Picture

Compiler's Role

Although the **hardware** generally resolves hazards to ensure correct execution, the **compiler** must understand the pipeline to achieve the best performance. Otherwise, unexpected stalls will reduce the performance of the compiled code. Compilers can reorder instructions to avoid hazards.

8 Flashcards

1. **Q: What is a ‘data hazard’ in the context of pipelined execution?**
A: It is an obstacle to pipelined execution where an instruction depends on the result of a previous instruction that is still in the pipeline.
2. **Q: In a pipeline diagram, how do data hazards visually manifest themselves?**
A: They appear as dependence lines that must go backward in time from when data is written to when it is read.
3. **Q: What is the primary hardware technique used to resolve most data hazards without stalling the pipeline?**
A: Forwarding, also known as bypassing, which passes the result directly from where it is computed to where it is needed.
4. **Q: For an ALU operation in the EX stage, the result of a previous instruction is needed at the _____ of the EX stage.**
A: beginning
5. **Q: How is a data hazard resolved if a register is read and written in the same clock cycle?**
A: The register file hardware is designed so the read gets the value that was written in the first half of the cycle, effectively performing an internal forward.
6. **Q: What does the notation ‘ID/EX.RegisterRs1’ specifically refer to in pipeline hazard detection?**
A: It refers to the number of the first source register (Rs1) for the instruction currently in the EX stage, held in the ID/EX pipeline register.
7. **Q: What is the first condition of a type ‘1a’ data hazard, also known as an EX hazard?**
A: The condition is $EX/MEM.RegisterRd = ID/EX.RegisterRs1$, where the previous instruction’s destination matches the current instruction’s first source.
8. **Q: What is the first condition of a type ‘2a’ data hazard, also known as a MEM hazard?**
A: The condition is $MEM/WB.RegisterRd = ID/EX.RegisterRs1$, where the destination of the instruction in the MEM stage matches the current instruction’s first source.
9. **Q: In the sequence ‘sub x2, x1, x3’ followed immediately by ‘and x12, x2, x5’, what specific type of hazard is detected?**
A: This is a type 1a hazard, as the and instruction needs the result of sub from the EX/MEM pipeline register.
10. **Q: In the sequence where ‘sub x2, x1, x3’ is followed two instructions later by ‘or x13, x6, x2’, what specific type of hazard is detected?**
A: This is a type 2b hazard, as the or instruction needs the result of sub from the MEM/WB pipeline register.
11. **Q: Why does the sequence ‘sub x2, ...’ followed by ‘add x14, x2, x2’ not present a data hazard that requires forwarding?**
A: Because the register file supplies the proper data during the ID stage of add, as sub will have completed its WB stage in the previous cycle.
12. **Q: What control signal must be checked in the forwarding logic to ensure that an instruction actually writes to a register?**
A: The RegWrite signal for the instruction in the EX or MEM stage must be checked.
13. **Q: Why must forwarding logic include the condition $RegisterRd \neq 0$?**
A: To prevent forwarding the result of an instruction that writes to register x0, which must always be treated as the value zero.
14. **Q: What hardware is added to the datapath to implement forwarding to the ALU?**
A: Multiplexors are added to each of the ALU inputs to select data from a register file read port or a pipeline register.
15. **Q: The forwarding unit, which generates control signals for the ALU input multiplexors, is located in which pipeline stage?**
A: The EX stage.

16. Q: What does the forwarding control signal ForwardA = 10 signify?

A: It instructs the ALU's 'A' input multiplexor to select the result from the prior instruction (via the EX/MEM pipeline register).

17. Q: What does the forwarding control signal ForwardB = 01 signify?

A: It instructs the ALU's 'B' input multiplexor to select the result from the MEM stage (via the MEM/WB pipeline register).

18. Q: In a sequence like 'add x1, x1, x2' followed by 'add x1, x1, x3', which result should be forwarded to the second add?

A: The result from the most recent instruction, which would be the first add instruction currently in the MEM stage.

19. Q: What check must be added to the MEM hazard detection logic to correctly handle back-to-back dependent instructions?

A: It must check that there is NOT a more recent EX hazard for the same register, ensuring the newest data is always used.

20. Q: What is the one specific case where forwarding cannot prevent a pipeline stall?

A: A load-use hazard, which occurs when an instruction tries to read a register immediately following a load instruction that writes to that same register.

21. Q: Why is forwarding insufficient to resolve a load-use data hazard?

A: The data from the load is not available until the end of the MEM stage (clock cycle 4), but the dependent instruction needs it at the beginning of its EX stage (also clock cycle 4).

22. Q: A _____ is an instruction that performs no operation and changes no state, used to resolve a load-use hazard.

A: nop (no operation)

23. Q: What is the function of the 'hazard detection unit' in a pipelined datapath?

A: It detects load-use hazards and stalls the pipeline to prevent incorrect execution.

24. Q: The hazard detection unit operates during the _____ stage to insert a stall between the load and the dependent instruction.

A: ID

25. Q: What is the precise condition the hazard detection unit checks for to initiate a stall?

A: if (ID/EX.MemRead and ((ID/EX.RegisterRd = IF/ID.RegisterRs1) or (ID/EX.RegisterRd = IF/ID.RegisterRs2)))

26. Q: When the hazard detection unit stalls the pipeline, which two registers are prevented from being updated?

A: The PC (Program Counter) and the IF/ID pipeline register.

27. Q: How is a nop or 'bubble' effectively inserted into the pipeline during a stall?

A: By deasserting all control signals (setting them to 0) in the EX, MEM, and WB control fields of the ID/EX pipeline register.

28. Q: What entity, other than the hardware itself, must understand the pipeline to achieve the best performance by avoiding stalls?

A: The compiler.

29. Q: The hazard detection unit controls a multiplexor that chooses between the real control values from the ID stage and a set of _____ to create a bubble.

A: all 0s

30. Q: To accommodate immediate values for instructions like addi, a 2:1 multiplexor is added to choose between the ForwardB multiplexor output and the _____.

A: signed immediate

31. Q: In a five-stage pipeline, if a load instruction is in the MEM stage (cycle 4), a dependent instruction would be in the _____ stage, creating a conflict.

A: EX

32. Q: Term: Stall

A: Definition: A technique to resolve a hazard by pausing the pipeline for one or more clock cycles.

33. **Q: The full logic for an EX hazard on the first ALU operand (Rs1) is: if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0) and (EX/MEM.RegisterRd = ____)) ForwardA = 10.**
A: ID/EX.RegisterRs1
34. **Q: The full logic for a MEM hazard on the second ALU operand (Rs2) is: if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0) and (MEM/WB.RegisterRd = ____)) ForwardB = 01.**
A: ID/EX.RegisterRs2
35. **Q: What is the effect of stalling on instructions that are in the fetch and decode stages?**
A: They are prevented from making progress; the instruction in IF is re-fetched and the instruction in ID is re-decoded in the next cycle.
36. **Q: Forwarding can resolve a data hazard between a store instruction's data source and a preceding ALU instruction because the data can be forwarded from the ____ or ____ stages to the MEM stage.**
A: EX, MEM
37. **Q: What is a data hazard in the context of pipelined execution?**
A: It is an obstacle where an instruction depends on the result of a previous instruction that has not yet completed its execution in the pipeline.
38. **Q: In a five-stage pipeline without forwarding, when is the result of an instruction written to the register file?**
A: During the middle of the fifth clock cycle, in the Write Back (WB) stage.
39. **Q: What problem arises if an instruction reads a register before a preceding instruction has written its new value back to it?**
A: The instruction will read the old, incorrect value from the register, leading to incorrect program execution.
40. **Q: The problem of a data hazard becomes apparent in a pipeline diagram when a dependence line goes ____ in time.**
A: backward
41. **Q: How does register file hardware design resolve a potential hazard when a register is read and written in the same clock cycle?**
A: It assumes the write occurs in the first half of the cycle and the read occurs in the second half, so the read gets the newly written value.
42. **Q: At what point in the pipeline is the result of an ALU operation, like sub, first available?**
A: At the end of the EX (Execute) stage.
43. **Q: At what point in the pipeline does an instruction like and or or need its operand values?**
A: At the beginning of its EX (Execute) stage.
44. **Q: What is the general principle of forwarding (or bypassing)?**
A: It is the process of taking a result directly from a pipeline register of a preceding instruction and sending it to a functional unit (like the ALU) for a subsequent instruction, bypassing the register file.
45. **Q: What does the notation ID/EX.RegisterRs1 refer to in pipeline analysis?**
A: It refers to the field in the ID/EX pipeline register that holds the number of the first source register (rs1).
46. **Q: What is the first condition for detecting a type 1a data hazard?**
A: The condition is EX/MEM.RegisterRd = ID/EX.RegisterRs1.
47. **Q: What is the first condition for detecting a type 2a data hazard?**
A: The condition is MEM/WB.RegisterRd = ID/EX.RegisterRs1.
48. **Q: In the sequence sub x2, x1, x3 followed by and x12, x2, x5, what type of hazard exists between them?**
A: A type 1a hazard, where EX/MEM.RegisterRd = ID/EX.RegisterRs1 = x2.
49. **Q: In the sequence sub x2, x1, x3 followed two instructions later by or x13, x6, x2, what type of hazard exists?**
A: A type 2b hazard, where MEM/WB.RegisterRd = ID/EX.RegisterRs2 = x2.

50. **Q: Why is there no data hazard between sub x2,... and add x14, x2, x2 if the add is three instructions later?**
A: Because the register file supplies the proper data during the ID stage of add, as the sub instruction will have completed its WB stage.
51. **Q: What additional check is necessary to prevent incorrect forwarding from instructions that do not write to a register?**
A: The hardware must check if the RegWrite control signal for the instruction in the EX or MEM stage is active (asserted).
52. **Q: Why must the hardware specifically prevent forwarding the result of an instruction whose destination is register x0?**
A: Because register x0 must always read as zero, and forwarding a non-zero result would violate this architectural rule.
53. **Q: What is the role of the forwarding unit?**
A: The forwarding unit contains the logic to detect data hazards and control the ALU input multiplexors to select the correct forwarded data.
54. **Q: In which pipeline stage does the forwarding control logic operate?**
A: It operates in the EX stage, as this is where the ALU forwarding multiplexors are located.
55. **Q: An EX hazard is a hazard between the instruction in the EX stage and the result of the instruction currently in the ____ stage.**
A: MEM
56. **Q: What control signal values are set for ForwardA if an EX hazard is detected for the first ALU operand (rs1)?**
A: The control value for ForwardA is set to 10.
57. **Q: A MEM hazard is a hazard between the instruction in the EX stage and the result of the instruction currently in the ____ stage.**
A: WB
58. **Q: What control signal values are set for ForwardB if a MEM hazard is detected for the second ALU operand (rs2)?**
A: The control value for ForwardB is set to 01.
59. **Q: If an operand has a data hazard from both the MEM stage and the WB stage, which result should be forwarded?**
A: The result from the MEM stage should be forwarded, as it is from the more recent instruction.
60. **Q: What is a load-use hazard?**
A: A situation where an instruction tries to read a register immediately following a ld (load) instruction that writes to that same register.
61. **Q: Why can't forwarding alone resolve a load-use data hazard?**
A: Because the data from the ld instruction is not available from memory until the end of the MEM stage, but the dependent instruction needs it at the beginning of its EX stage one cycle earlier.
62. **Q: What hardware unit is required specifically to handle load-use hazards?**
A: A hazard detection unit.
63. **Q: In which pipeline stage must the hazard detection unit operate to stall the pipeline for a load-use hazard?**
A: It must operate during the ID (Instruction Decode) stage.
64. **Q: What is the condition checked by the hazard detection unit to initiate a stall?**
A: if (ID/EX.MemRead and ((ID/EX.RegisterRd = IF/ID.RegisterRs1) or (ID/EX.RegisterRd = IF/ID.RegisterRs2)))
65. **Q: What is the primary action taken by the hardware when a pipeline stall is initiated?**
A: It prevents the PC register and the IF/ID pipeline register from changing for one clock cycle.
66. **Q: What is a 'nop' instruction in the context of pipelining?**
A: A nop (or bubble) is an instruction that does no operation to change state, effectively inserted into the pipeline to create a delay.

67. **Q: How is a nop or bubble created and inserted into the pipeline by the hazard detection unit?**
A: By deasserting all the control fields (setting them to 0) in the ID/EX pipeline register for the stalled instruction.
68. **Q: What is the effect of a stall on the instruction in the ID stage and the instruction in the IF stage?**
A: Both instructions are delayed; the ID stage instruction repeats decoding, and the IF stage instruction is re-fetched from the same PC value.
69. **Q: While the compiler usually relies on hardware to resolve hazards, how can a compiler improve performance?**
A: The compiler can reorder instructions to avoid sequences that would cause stalls, thus improving performance.
70. **Q: What is the purpose of the ID/EX.MemRead signal in the load-use hazard detection logic?**
A: It identifies that the instruction currently in the EX stage is a load, which is the only instruction that can cause this type of stall.
71. **Q: What does a control value of 00 for the ForwardA or ForwardB multiplexor signify?**
A: It signifies that no forwarding is needed for that input, and the value should come from the register file via the ID/EX pipeline register.